Guía de Adaptación de Stack Tecnológico

Este Kit Pedagógico tiene un núcleo metodológico stack-agnóstico y una capa de ejemplos prácticos que usan React + Supabase + Deno/TypeScript. Este documento le indica al docente qué mantener tal cual, qué adaptar y cómo hacerlo para su stack específico.

Nivel 1: No tocar (Metodología pura)

Estos archivos son stack-agnósticos. Funcionan igual sin importar la tecnología. No requieren modificación.

Archivo / Directorio	Por qué es agnóstico
`01_Syllabus/especificacion_metodologia.pdf`	Define los 6 pilares (PCA, Scaffolding, Contratos, etc.). Son conceptos de Ingeniería de Software, no de una librería.
`01_Syllabus/cronograma.pdf`	Timing y estructura de los talleres. Aplica a cualquier asignatura.
`01_Syllabus/propuesta_integracion_curricular.pdf`	Mapeo conceptual de tradicional a nueva metodología.
`02_Guia_Docente/INDICE_DOCENTE_Y_MAPA.pdf`	Mapa de navegación del kit.
`03_Material_Estudiante/01_Plantillas_y_Workflows/*`	Prompts de Analista y Arquitecto. Son instrucciones para la IA, no código.
`03_Material_Estudiante/02_Fundamentos_y_Comunicacion/*`	Conceptos (PCA, Vibe Coding, Comunicación con IA). Aplican a cualquier lenguaje.
`03_Material_Estudiante/04_Ejercicios_Microtasks/*`	Ejercicios conceptualizados (HTML/CSS básico).
`04_Evaluaciones/rubrica_evaluacion_arquitecto.pdf`	Rúbrica de 6 dimensiones. Evalúa metodología, no tecnología.
`05_Presentaciones_Autoexplicativas/*`	Material de estudio del estudiante.
`06_Base_Conocimiento_Docente/*`	Base de conocimiento del docente.

Nivel 2: Adaptar ejemplos (Mismo concepto, diferente sintaxis)

Estos archivos contienen ejemplos de código que deben reescribirse para el stack del docente. La **estructura y los conceptos** se mantienen; solo cambia la **sintaxis**.

Archivo	Qué adaptar
`03_Arquitectura_y_Criterio/convenciones_y_patrones_base.pdf`	Los ejemplos de DI, Singleton y Repository están en TypeScript. Reescribir en el lenguaje del curso.
`03_Arquitectura_y_Criterio/guia_scaffolding_arquitectura.pdf`	El árbol de carpetas usa `/services`, `/components`. Adaptar a la convención del stack (ej. `src/main/java/`).
`03_Arquitectura_y_Criterio/guia_testing_y_calidad.pdf`	Los ejemplos de Jest/Vitest pueden cambiarse a JUnit, pytest, o el framework de testing del stack.
`04_Evaluaciones/Ejercicios_Nivelacion/*`	Reescribir los snippets de código usando el stack del curso. Mantener las preguntas y la pauta de evaluación.
`06_Base_Conocimiento_Docente/.../01_ejercicio_vibe_coding_vs_pca.pdf`	Demo en vivo. Reescribir los ejemplos de código para el stack usado en clase.

Nivel 3: Reemplazar (Dependiente del stack)

Estos archivos están atados a React + Supabase + Deno y necesitan ser reescritos. Los conceptos se mantienen; la implementación cambia.

Archivo	Qué reemplazar
`03_Arquitectura_y_Criterio/guia_ciberseguridad_y_entorno.pdf`	Adaptar a vulnerabilidades del stack del curso (ej. en Java: SQL Injection; en Python: CSRF, Django ORM).
`03_Arquitectura_y_Criterio/guia_criterio_arquitectonico.pdf`	Adaptar capacidades del stack (ej. en Java: calcular en el Stored Procedure; en Python: usar Django ORM con aggregate).
`07_Presentaciones_Clase/*`	Reescribir los snippets manteniendo la estructura de diapositivas.

Ejemplo: Cómo se vería el Taller 3 en Java/Spring Boot

Original (Supabase/TypeScript):

```
// Singleton para Supabase
const supabase = createClient(SUPABASE_URL, SUPABASE_KEY);
```

Adaptado (Java/Spring Boot):

```
// Singleton via Spring IoC (ya es singleton por defecto)
@Service
public class UserService {
    private final UserRepository repo;

    @Autowired
    public UserService(UserRepository repo) {
        this.repo = repo; // Spring inyecta la única instancia
    }
}
```

El concepto (Singleton, no crear conexiones nuevas) es idéntico. Solo cambia la implementación.

Ejemplo: Cómo se vería el Taller 3 en Python/Django

Original (Supabase RLS):

```
CREATE POLICY "users_select_own" ON users
  FOR SELECT USING (auth.uid() = id);
```

Adaptado (Django):

```
# Django ya protege contra SQL injection via ORM
# El equivalente a RLS es el permissions system
class User(models.Model):
    name = models.CharField(max_length=100)

    class Meta:
        permissions = [
            ("view_own_profile", "Can view own profile"),
        ]
```

El concepto (el usuario solo ve sus propios datos) es idéntico. El mecanismo de protección cambia.

Recomendación final

El docente que adapte este kit debe seguir este orden:

1. **Leer primero** `01_Syllabus/especificacion_metodologia.pdf` para entender los 6 pilares.
2. **Identificar su stack** y revisar qué archivos del Nivel 2 y 3 necesita modificar.
3. **Reescribir los ejemplos de código** manteniendo la misma estructura narrativa y los mismos conceptos.
4. **No modificar** los archivos del Nivel 1. Son el núcleo del kit y funcionan para cualquier stack.